LearnDeep

Technical Reference

Subhodeep Mondal

October 24, 2026

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 Scope . . . 1
- 1.4 Organization of This Document . . . 2

2 Deep Learning Framework from Scratch **3**
- 2.1 Tensor 101 . . . 3
  - 2.1.1 Why Do We Need Tensors? . . . 3
  - 2.1.2 A Different Dimension Convention . . . 4
- 2.2 Working with Tensors . . . 4
- 2.3 How Should the Mathematics Be Implemented? . . . 5
- 2.4 Testing and Numerical Verification . . . 6
- 2.5 Closing Remarks . . . 6

3 Tensor Operations and Kernel Dispatching **7**
- 3.1 Introduction . . . 7
- 3.2 Basic Element-Wise Operations . . . 8
- 3.3 Matrix Multiplication . . . 8
- 3.4 Broadcasting . . . 9
- 3.5 Greater-Than-Zero Operation . . . 10
- 3.6 Kernel Dispatching . . . 10
- 3.7 Testing and Validation . . . 11
- 3.8 Closing Remarks . . . 11

4 Kernels **12**

5 Automatic Differentiation **13**
- 5.1 Autograd 101 . . . 13
- 5.2 Forward Propagation and Backward Propagation . . . 14
- 5.3 Why Reverse-Mode Automatic Differentiation? . . . 14
- 5.4 Constructing the Gradient Graph . . . 15
- 5.5 Visualizing the Forward and Backward Graphs . . . 16
- 5.6 Triggering Automatic Differentiation . . . 17
- 5.7 Testing and Validation . . . 17
- 5.8 Closing Remarks . . . 17

6 Neural Network Components: Layers **19**
- 6.1 Layer . . . 19
- 6.2 Layer: Dense . . . 20
  - 6.2.1 Dense: Constructor . . . 21
  - 6.2.2 Dense: Overloaded () operator . . . 21

6.2.3 Dense: forward() backward() calculate() . . . 22
6.2.4 Dense in practice . . . 23

7 Neural Network Components: Losses **25**
7.1 Loss Functions: Mean Squared Error (MSE) . . . 26
7.1.1 MSE: Differentiation . . . 26
7.1.2 MSE: Putting things together . . . 26
7.2 Loss Function: Categorical Cross Entropy (CCE) . . . 27
7.2.1 CCE: Differentiation . . . 27
7.2.2 CCE: Putting things together . . . 27
7.2.3 CCE: In practice . . . 28

8 Neural Network Components: Optimizers **29**
8.1 Gradient Descent: GD . . . 30
8.1.1 Batch Gradient Descent: BGD . . . 30
8.1.2 Mini Batch Gradient Descent: MGD . . . 30
8.1.3 Stocastic Gradient Descent: SGD . . . 31
8.2 Stocastic Gradient Descent . . . 31
8.3 SGD with momentum . . . 32

9 Neural Network Model Training and Inference **33**
9.1 The Model Class . . . 34
9.1.1 Model Construction and Compilation . . . 34
9.1.2 Model Training . . . 35
9.1.3 Model Inference . . . 36
9.2 Numerical Verification . . . 37

Chapter 1

Introduction

1.1 Overview

LearnDeep is an educational deep learning framework developed from scratch to explore the fundamental concepts, algorithms, and engineering principles behind modern deep learning systems.

The primary objective of this project is to understand how a deep learning framework operates internally, from tensor computations and numerical kernels to automatic differentiation, neural network layers, and model training.

Rather than treating a deep learning framework as a black box, LearnDeep aims to make its underlying mechanisms explicit through a modular implementation and a focus on mathematical correctness.

1.2 Motivation

Modern deep learning frameworks provide high-level interfaces for constructing, training, and evaluating neural networks. However, many of the mechanisms responsible for these operations remain hidden behind their public APIs.

Developing LearnDeep provides an opportunity to investigate these mechanisms directly, including:

- Tensor representation, indexing, and memory ownership.
- Numerical operations, broadcasting, and CPU and GPU kernel execution.
- Computational graphs and reverse-mode automatic differentiation.
- Neural network layers, loss functions, and gradient propagation.
- Model training, optimization, and execution management.

Implementing these components from the ground up provides practical insight into the relationship between mathematical operations, software architecture, and hardware-level computation.

1.3 Scope

LearnDeep is intended primarily as a learning and experimentation platform. Its implementation prioritizes understanding, modularity, and correctness while exploring the design of a deep learning framework.

The technical reference documents the framework's architecture, public interfaces, mathematical foundations, and important implementation decisions.

It is organized around the major components of the system, allowing readers to understand both how individual components work and how they interact during model execution and training.

1.4 Organization of This Document

The remainder of this document is organized as follows:

- **Tensor System:** Tensor representation, shape semantics, data layout, and ownership.
- **Operations and Kernels:** Numerical operations, broadcasting, and backend execution.
- **Automatic Differentiation:** Computational graphs, gradient propagation, and backward execution.
- **Neural Network Components:** Layers, loss functions, and model construction.
- **Training:** Forward execution, optimization, and training workflows.

These sections describe the design and behavior of the implemented system, including relevant mathematical definitions and validation methods.

Chapter 2

Deep Learning Framework from Scratch

2.1 Tensor 101

A tensor is a multidimensional array. Scalars contain a single value, vectors extend this idea to one dimension, and matrices extend it to two dimensions. Tensors generalize the same concept to an arbitrary number of dimensions.

Einstein did not invent tensors, but his work helped popSularize their use in physics.

2.1.1 Why Do We Need Tensors?

To do mathematics, of course.

At a basic level, a tensor implementation needs a block of memory to store its elements and metadata describing the size of each dimension. In this implementation, the elements are stored in a contiguous memory allocation, while the dimensions are represented separately.

A simplified version of the class looks like this:

```
template<typename T>
class tensor {
    T* data;                    // Manual memory management.
    size_t no_of_dimensions;
    size_t* dimensions;

public:
    tensor();
    ~tensor();

    tensor add(tensor b);
    tensor matmul(tensor b);
};
```

Listing 2.1: A simplified tensor class

No `std::unique_ptr`, no `std::vector`. As the joke goes: *"Hum toh mard hai, Jaskirat. We manage our own memory."*

Of course, manual memory management also means that allocation, deallocation, copying, and ownership must be handled carefully. This is one of the engineering trade-offs involved in building the underlying storage abstraction yourself.

For example,

```
tf::tensor A(2, 3, 4);
```

Listing 2.2: Creating a three-dimensional tensor

creates a three-dimensional tensor with dimensions $(2, 3, 4)$. It contains 2 elements along the first dimension, 3 along the second, and 4 along the third, for a total of $2 \times 3 \times 4 = 24$ elements.

Simple enough. No rocket science.

2.1.2 A Different Dimension Convention

There is an important design choice in LearnDeep: its dimension ordering differs from the convention commonly encountered in NumPy, TensorFlow, and PyTorch.

In LearnDeep, the leftmost dimension is treated as the primary dimension. Consequently, a tensor described by dimensions $(2, 3, 4)$ follows the framework's own axis convention and should not be assumed to have the same interpretation as a tensor with that shape in another library.

When comparing shapes or transferring data between frameworks, this difference must be taken into account. Changing the convention later would require careful refactoring of operations, kernels, and code that depends on the current layout.

For now, this is the convention used by LearnDeep.

2.2 Working with Tensors

Having a tensor abstraction is useful only if we can perform computations with it. Tensor operations provide the basic mathematical building blocks used throughout the framework.

For example, two tensors can be created, populated with data, and added together:

```
tf::tensor A, B, C;

A.tf_create(2, 3, 4);
B.tf_create(2, 3, 4);

A.tensor_of(data_1);
B.tensor_of(data_2);

C = A.add(B);
```

Listing 2.3: Adding two tensors

Matrix multiplication can be expressed in a similar way:

```
A.tf_create(2, 3);
B.tf_create(4, 2);

C = A.matmul(B);

// Result dimensions: (4, 3)
```

Listing 2.4: Matrix multiplication using LearnDeep's dimension convention

The result dimensions shown above follow LearnDeep's custom axis convention. They differ from the shape notation readers may expect when working with NumPy, so the framework's own conventions must be kept in mind when interpreting the operation.

These examples illustrate the public interface. The actual arithmetic is performed by the underlying operation and kernel implementation.

2.3 How Should the Mathematics Be Implemented?

BLAS, BLIS, LAPACK, cuBLAS, Eigen...

Those libraries provide powerful implementations of numerical operations. However, the purpose of this project is to understand how such computations work internally, so I decided to implement the kernels myself.

One possible approach would be to place the implementation of every operation directly inside the tensor class:

```
class tensor {
    tensor add(tensor input_b) {
        tensor output;

        // Iterate over the elements and add them.

        return output;
    }
};
```

Listing 2.5: A direct implementation inside the tensor class

This is simple to understand, but it mixes tensor storage and interface responsibilities with the implementation of individual operations.

Instead, LearnDeep separates operations from the tensor abstraction. This provides a place to organize computation and, as the framework grows, to support features such as:

1. **Kernel dispatching:** selecting an appropriate implementation, such as a basic CPU, AVX2, or GPU kernel.
2. **Kernel registration:** organizing the available kernel implementations.
3. **Automatic differentiation:** associating operations with the information needed to calculate gradients during backpropagation.

A simplified illustration of this idea is shown below:

```
class ops {
public:
    virtual void compute() = 0;
    virtual void kernel_dispatcher() = 0;
};

class add : protected ops {
public:
    void compute() override;
    void kernel_dispatcher() override;
};
```

Listing 2.6: An illustrative abstraction for operations

This is a conceptual example rather than a complete representation of the final implementation. The design can evolve as additional operations and execution backends are introduced.

The important idea is to keep the tensor abstraction, operation definitions, and kernel execution strategy sufficiently separate. This gives the framework room to grow without forcing every new implementation detail into the tensor class itself.

2.4 Testing and Numerical Verification

Once the basic abstractions and operations are in place, the next step is testing.

For this project, NumPy serves as an independent numerical reference. I generate test data, calculate expected results using NumPy, and compare those results against the output produced by LearnDeep.

This approach helps identify discrepancies in tensor operations and provides a practical way to verify that the framework's computations behave as expected.

The tests are an important part of the development process: successful compilation alone does not establish that an operation is numerically correct.

2.5 Closing Remarks

This is the starting point of LearnDeep: a tensor abstraction, a growing set of mathematical operations, and an implementation strategy that leaves room for multiple kernel backends and automatic differentiation.

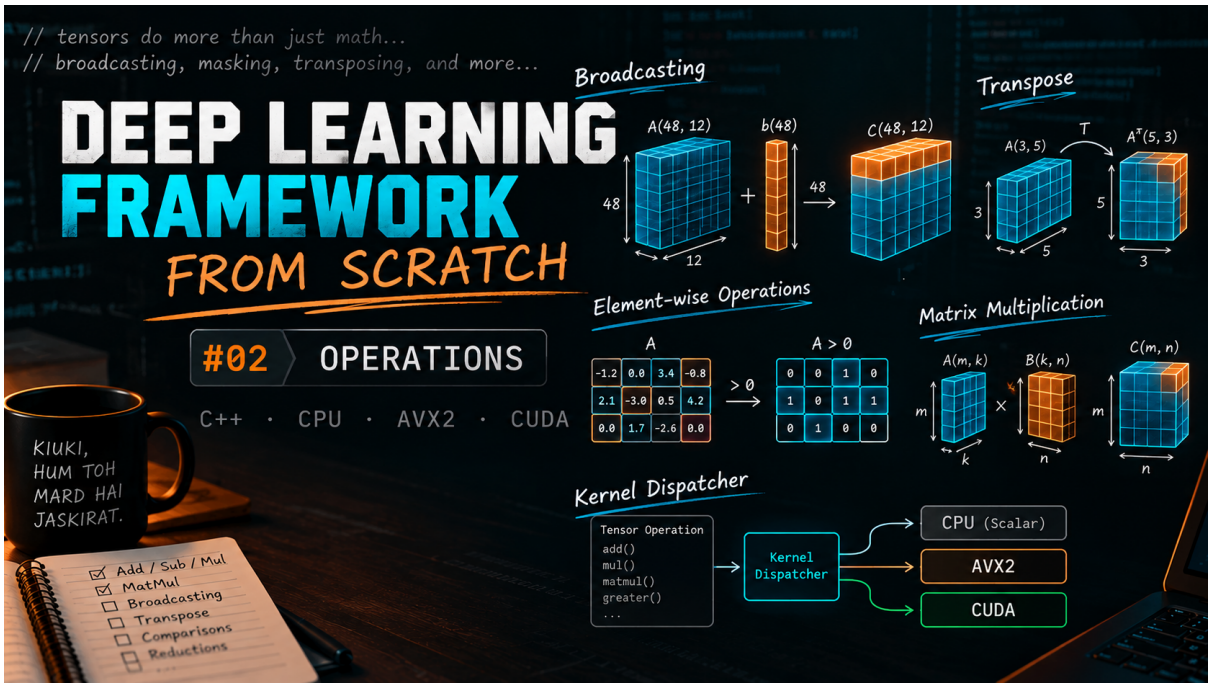
The framework will become more capable as these foundations are extended and connected. For now, the priority is to build the core carefully, test it against independent reference results, and understand the engineering decisions behind each component.

There is plenty left to do.

To be continued...

Chapter 3

Tensor Operations and Kernel Dispatching



3.1 Introduction

In the previous chapter, we introduced the motivation behind LearnDeep and established a basic tensor abstraction. We now turn to the operations that make tensors useful.

Tensor operations are not limited to mathematical calculations. They can also transform data, change how it is interpreted, apply logical conditions, or move values between locations. In a deep learning framework, these operations form the building blocks from which more complex computations are assembled.

This chapter explores several important operations and the design decisions involved in implementing them efficiently.

3.2 Basic Element-Wise Operations

Operations such as `add()`, `sub()`, and `mul()` apply a calculation independently to corresponding elements of their input tensors, provided their shapes are compatible.

At a basic level, an element-wise operation can be represented as:

```
for (each element i) {
    output[i] = ops(input_a[i], input_b[i]);
}
```

Listing 3.1: A generic element-wise operation

When tensor data is stored contiguously, iterating through adjacent elements can provide a straightforward memory access pattern. On a GPU, adjacent threads accessing adjacent memory locations can also enable coalesced memory accesses, depending on the mapping of threads to data.

One way to expose more independent operations is loop unrolling. For example, a loop can process eight elements per iteration:

```
for (each row) {
    for (each group of 8 elements) {
        output[i]     = input_a[i]     + input_b[i];
        output[i + 1] = input_a[i + 1] + input_b[i + 1];
        output[i + 2] = input_a[i + 2] + input_b[i + 2];
        output[i + 3] = input_a[i + 3] + input_b[i + 3];
        output[i + 4] = input_a[i + 4] + input_b[i + 4];
        output[i + 5] = input_a[i + 5] + input_b[i + 5];
        output[i + 6] = input_a[i + 6] + input_b[i + 6];
        output[i + 7] = input_a[i + 7] + input_b[i + 7];
    }

    // Handle any remaining elements.
}
```

Listing 3.2: An illustrative unrolled element-wise loop

Loop unrolling and vectorization are related but distinct techniques. Unrolling reduces loop-control overhead and exposes multiple independent operations, while vectorization performs multiple element operations using a single vector instruction.

LearnDeep uses `float64` for these kernels. A 256-bit AVX2 vector can therefore hold four double-precision values, allowing an appropriate vectorized kernel to process four such values per vector instruction.

The actual performance benefit depends on the compiler, memory access pattern, data size, and implementation. Unrolling alone does not guarantee vectorization, so the two techniques should be considered separately.

3.3 Matrix Multiplication

Matrix multiplication is one of the central operations in deep learning. Dense layers, many attention computations, and numerous other neural network operations rely on matrix products.

The conventional scalar implementation uses three nested loops. For matrices with compatible dimensions, the computation can be expressed as follows:

```
for (unsigned i = 0; i < m; i++) {
    for (unsigned j = 0; j < n; j++) {
        C[i][j] = 0;

```

```
            for (unsigned k = 0; k < l; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
}
```

Listing 3.3: A straightforward matrix multiplication loop

Although this implementation expresses the mathematics clearly, its memory access pattern can be inefficient for row-major storage.

In the innermost loop, the code accesses consecutive elements from a row of A, but accesses a column of B. Moving between successive elements of that column jumps across rows, potentially reducing cache efficiency.

We can change the loop ordering to improve the access pattern:

```
// Initialize C to zero before these loops.

for (unsigned i = 0; i < m; i++) {
    for (unsigned j = 0; j < l; j++) {
        for (unsigned k = 0; k < n; k++) {
            C[i][k] += A[i][j] * B[j][k];
        }
    }
}
```

Listing 3.4: Matrix multiplication with interchanged loops

In this arrangement, the innermost loop traverses consecutive elements in a row of B and updates consecutive elements in a row of C. This can improve memory locality compared with the original loop order.

The output matrix must be initialized to zero before accumulation. The dimensions and loop bounds must also match the matrix multiplication convention used by the implementation.

Can we do better? Certainly. Techniques such as cache blocking, loop tiling, vectorization, and GPU tiling can further improve performance. These optimizations deserve a deeper discussion of their own and will be explored separately.

3.4 Broadcasting

Consider the following operation using LearnDeep's dimension convention:

```
tensor A(48, 12);
tensor B(48);

tensor C = A.add(B);
```

Listing 3.5: An example of broadcasting

What should the dimensions of C be?

Under LearnDeep's convention, the 48-element tensor aligns with the first dimension of A, so the resulting tensor has dimensions $(48, 12)$. Each value in B is reused along the second dimension.

This differs from the conventional shape interpretation used by libraries such as NumPy, PyTorch, and TensorFlow. Under their standard broadcasting rules, shapes $(48, 12)$ and $(48,)$ are not directly compatible. The axis convention must therefore be taken into account when comparing behavior across frameworks.

Broadcasting allows tensors with compatible dimensions to participate in an operation without explicitly copying the smaller tensor to match the larger one.

A common example appears in dense layers. Matrix multiplication produces outputs for a batch of samples, after which a bias vector is added to each sample. Instead of manually duplicating the bias vector for every sample, broadcasting allows the operation to reuse its values.

The main challenge is not the addition itself, but determining shape compatibility and mapping each output element to the appropriate input elements. In an ordinary element-wise operation, corresponding elements can be accessed directly. With broadcasting, one input element may contribute to multiple output elements.

Correct broadcasting behavior is therefore essential when composing tensor operations into higher-level layers.

3.5 Greater-Than-Zero Operation

A greater-than-zero operation evaluates each tensor element and produces 1 when the element is positive and 0 otherwise.

For example:

```
input:  [-2.5, 0.0, 3.7, -1.2]
output: [ 0.0, 0.0, 1.0,  0.0]
```

Listing 3.6: Illustrative input and output for a greater-than-zero operation

Although simple, this operation can be useful in neural network computations. Consider the Rectified Linear Unit (ReLU) activation function:

$$\mathrm{ReLU}(x) = \max(0, x)$$

During the forward pass, negative values become zero while positive values pass through unchanged.

Away from zero, its derivative is:

$$\mathrm{ReLU}'(x) = \begin{cases} 0 & x < 0, \\ 1 & x > 0. \end{cases}$$

A greater-than-zero mask can help implement this derivative during backpropagation. The derivative at exactly zero requires a chosen convention, and the mask itself is only one part of the backward computation.

This illustrates an important point: operations that appear trivial in isolation can become useful primitives when applied across entire tensors or combined into larger computational graphs.

Their role becomes even clearer when we examine automatic differentiation and graph execution.

3.6 Kernel Dispatching

So far, we have discussed what several tensor operations compute. But an operation's mathematical definition is separate from the implementation that executes it.

For example, tensor addition could be implemented using a scalar CPU kernel, an AVX2 vectorized kernel, or a CUDA kernel running on a GPU. These implementations aim to perform the same operation, but use different execution resources and strategies.

The kernel dispatcher provides a place to select an appropriate implementation rather than repeating backend-selection logic inside every operation.

Conceptually, the flow is:

Tensor operation $\longrightarrow$ Kernel dispatcher $\longrightarrow$ Selected implementation

Possible implementations include:

- **Scalar CPU:** a basic implementation suitable as a reference or fallback.
- **AVX2:** a CPU implementation that uses vector instructions where appropriate.
- **CUDA:** a GPU implementation for operations with supported CUDA kernels.

The dispatcher must consider the requested execution backend and whether an implementation of the operation is available. If the requested kernel is unavailable, the framework can use a supported fallback where appropriate; a GPU request does not imply that every operation already has a CUDA implementation.

Separating operation definitions from kernel selection reduces duplicated logic and provides a natural extension point as new operations and backends are introduced.

In short, the operation describes *what* should be computed, while the dispatcher helps determine *which implementation* will compute it.

3.7 Testing and Validation

The behavior of tensor operations must be verified rather than assumed. LearnDeep uses test cases that compare its computed results against independently generated reference values, including values calculated with NumPy.

These tests help identify discrepancies in element-wise operations, matrix multiplication, broadcasting, and other supported computations.

3.8 Closing Remarks

Tensor operations form the computational foundation of LearnDeep. Their implementation involves more than translating mathematical formulas into loops: memory layout, cache locality, vectorization, broadcasting rules, and execution-backend selection all influence how the framework behaves.

Separating operations from their kernel implementations also establishes a foundation for supporting multiple execution backends without duplicating the same dispatch logic throughout the library.

This chapter has introduced several of these ideas through basic element-wise operations, matrix multiplication, broadcasting, logical masks, and kernel dispatching. More advanced kernel optimizations, including cache blocking and tiled GPU execution, remain natural areas for further exploration.

There is plenty more to build.

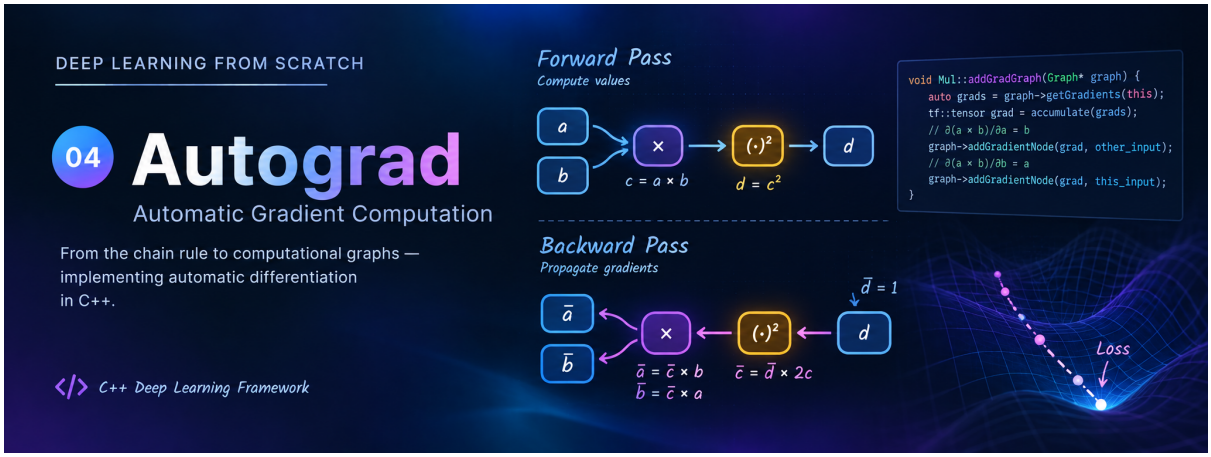
To be continued...

Chapter 4

Kernels

Chapter 5

Automatic Differentiation



5.1 Autograd 101

Automatic gradient computation, commonly called *automatic differentiation* or *autodiff*, is a technique for calculating derivatives of a computation with respect to its inputs and parameters.

Consider the function

$$y = x^2 + 3x.$$

Its derivative with respect to x is

$$\frac{dy}{dx} = 2x + 3.$$

For a simple expression, deriving the derivative manually is straightforward. However, neural networks can contain many tensor operations distributed across multiple layers. Manually deriving and implementing every possible derivative for every possible model would quickly become impractical.

Automatic differentiation addresses this problem by applying derivative rules to the operations performed during a computation and combining them using the chain rule.

It is important to distinguish autodiff from symbolic differentiation and numerical differentiation. Symbolic differentiation manipulates mathematical expressions to produce derivative expressions, while numerical differentiation estimates derivatives using numerical approximations, such as finite differences. Automatic differentiation instead propagates derivative information through the computation itself.

5.2 Forward Propagation and Backward Propagation

Consider a small example:

$$a = 2, \qquad b = 3,$$
$$c = a + b, \qquad d = c^2.$$

During the forward pass, the intermediate and final values are calculated:

$$c = 5, \qquad d = 25.$$

Suppose we want to calculate the derivatives of d with respect to a and b. Applying the chain rule gives

$$\frac{\partial d}{\partial c} = 2c = 10.$$

Since $c = a + b$,

$$\frac{\partial c}{\partial a} = 1, \qquad \frac{\partial c}{\partial b} = 1.$$

Therefore,

$$\frac{\partial d}{\partial a} = 10, \qquad \frac{\partial d}{\partial b} = 10.$$

The forward pass calculates the values of the computation, while the backward pass calculates how changes in those values influence the selected output.

This distinction is fundamental to training neural networks: the forward pass produces predictions and the backward pass computes the gradients needed to update trainable parameters.

5.3 Why Reverse-Mode Automatic Differentiation?

Automatic differentiation can be performed in different modes. Two commonly discussed approaches are forward-mode and reverse-mode differentiation.

- **Forward mode** propagates derivative information alongside the forward computation.
- **Reverse mode** first evaluates the computation and then propagates gradients backward from the output.

Reverse mode is especially useful when a function has many inputs but relatively few outputs. Neural networks often have a large number of trainable parameters and a scalar loss, making reverse-mode differentiation a natural choice for computing the gradients required during training.

LearnDeep builds on its graph-mode execution to support this process. Since the forward operations and their dependencies are recorded in a computational graph, the framework can traverse those dependencies in reverse order and construct the corresponding gradient computation.

5.4 Constructing the Gradient Graph

The autodiff implementation in LearnDeep follows three broad steps:

1. **Recording dependencies:** operations establish relationships between their input and output tensors in the computational graph.
2. **Propagating gradients:** starting from the selected output, the framework visits operations in reverse dependency order and applies the derivative rule associated with each operation.
3. **Accumulating gradients:** when a tensor contributes to multiple downstream operations, the gradient contributions from those paths must be added together.

Each operation is responsible for registering the backward computation corresponding to its derivative rule. In LearnDeep, this responsibility is represented by an `addGradGraph()` routine that receives a pointer to the gradient graph.

The following simplified pseudocode illustrates the idea for a multiplication operation:

```
void Mul::addGradGraph(Graph* graph) {
    // Retrieve gradients arriving from downstream operations.
    std::vector<tf::tensor> incoming_gradients =
        graph->getGradients(this);

    // Accumulate incoming gradients where necessary.
    // ...

    // For z = a * b:
    // dz/da = b
    // dz/db = a

    // Register backward operations that multiply the
    // incoming gradient by the opposite input.
    // ...

    // Store the resulting gradient nodes for each input.
}
```

Listing 5.1: Conceptual gradient-graph construction for multiplication

The example is intentionally simplified. It illustrates the responsibilities of the routine rather than specifying the complete implementation or its ownership details.

For a multiplication $z = ab$, the local derivatives are

$$\frac{\partial z}{\partial a} = b, \qquad \frac{\partial z}{\partial b} = a.$$

If the incoming gradient from downstream computation is $\bar{z}$, the gradients propagated to the inputs are

$$\bar{a} = \bar{z}b, \qquad \bar{b} = \bar{z}a.$$

Here, the bar notation represents the gradient propagated to a tensor. In a computational graph with multiple downstream paths, these contributions must be accumulated before the gradient is propagated further upstream.

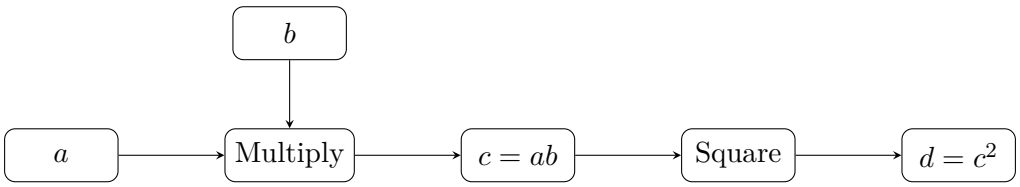
This is how the framework builds the backward computation associated with each supported operation.

5.5 Visualizing the Forward and Backward Graphs

Consider the computation

$$c = ab, \qquad d = c^2.$$

The forward graph describes how the output is calculated from the inputs:



The graph begins with the inputs a and b. Multiplication produces c, which is then squared to produce d.

For reverse-mode differentiation, we seed the output gradient with

$$\bar{d} = \frac{\partial d}{\partial d} = 1.$$

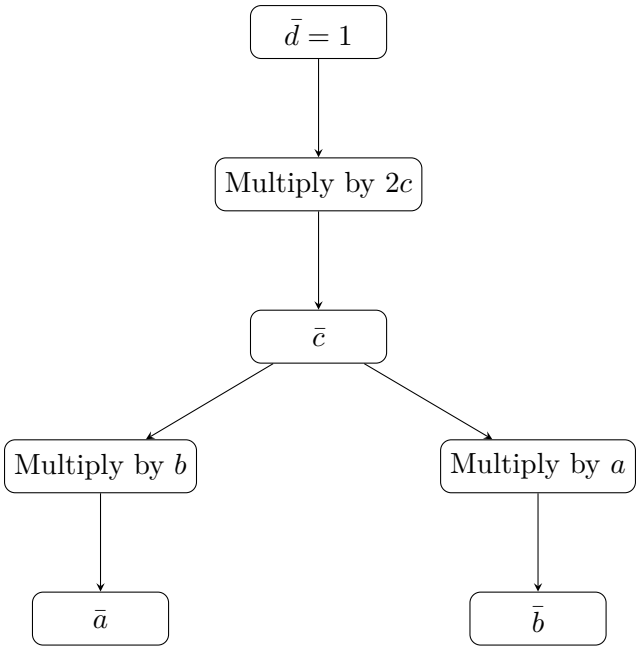
For the square operation,

$$\bar{c} = \bar{d}\, 2c.$$

For the multiplication operation,

$$\bar{a} = \bar{c}b, \qquad \bar{b} = \bar{c}a.$$

The backward computation therefore follows the derivative rules of the forward operations, rather than simply repeating the original operations in reverse order.



The forward graph calculates values; the backward graph calculates gradients. Keeping these roles distinct helps explain how reverse-mode autodiff turns a recorded computation into the gradient calculations required for learning.

5.6 Triggering Automatic Differentiation

The public API brings the graph-construction and gradient-execution steps together. The following example illustrates the intended workflow:

```
{
    tf::graph_context ctx;

    // Forward operations are recorded here.
    // ...

    ctx.initialize_gradient(); // Construct the gradient graph
    ctx.compute_gradient();    // Execute the gradient graph

    tf::tensor input_a_grad =
        ctx.get_gradient(input_a); // Retrieve the input gradient
}
```

Listing 5.2: Triggering automatic differentiation

Conceptually, the process proceeds as follows:

1. The autodiff API is invoked within a graph context.
2. The graph context obtains the active graph session.
3. The graph's `createGradientGraph()` routine traverses the operation nodes using depth-first search and registers the backward computations.
4. The gradient graph is executed, and the resulting gradients can be retrieved for the corresponding tensors.

With this mechanism, LearnDeep can automatically propagate gradients through the operations supported by its computational graph. The exact set of supported gradients depends on the derivative rules implemented for those operations.

5.7 Testing and Validation

Automatic differentiation must be tested independently of the forward computation. A correct forward result does not guarantee that the gradients are correct.

Useful tests include comparing analytically known derivatives against the framework's calculated gradients, checking gradient accumulation when a tensor feeds multiple downstream operations, and verifying that gradients propagate correctly through compositions of supported operations.

5.8 Closing Remarks

Automatic differentiation connects the forward computations of a neural network to the gradients required for learning. By recording operation dependencies, constructing backward computations, and accumulating gradient contributions, LearnDeep can differentiate supported computations without requiring the user to manually derive the full gradient of every model.

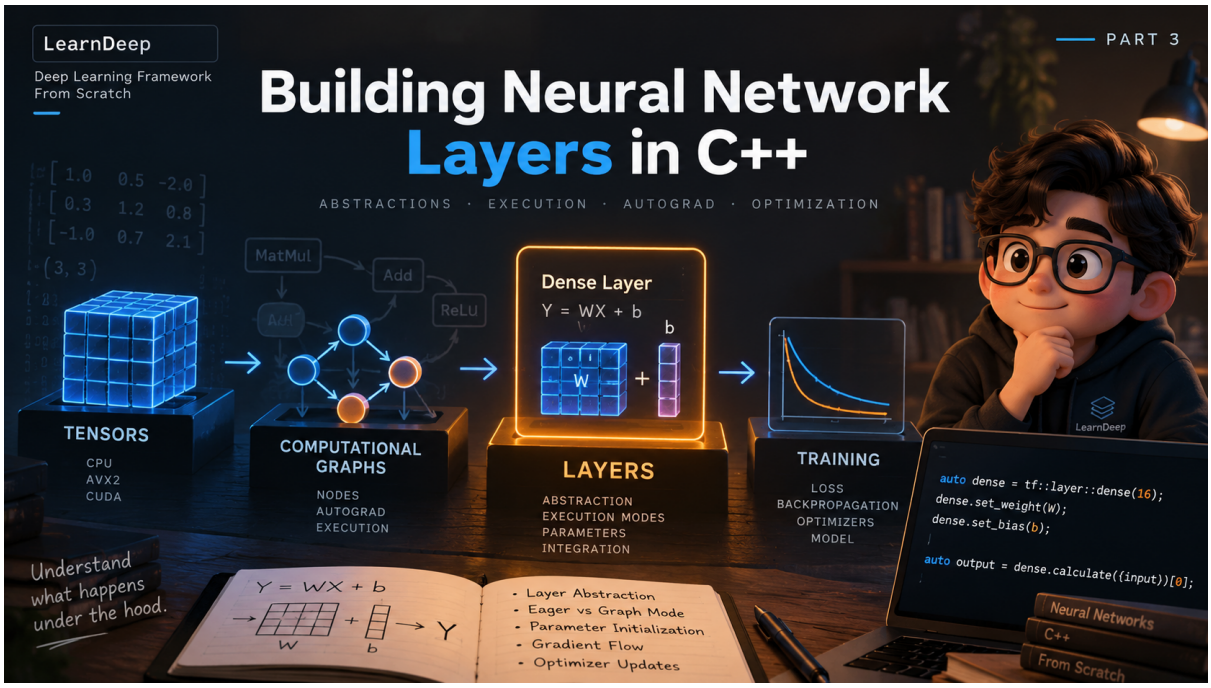
This implementation builds on the framework's computational graph and operation abstractions. Each operation supplies its derivative behavior, while the graph coordinates the backward traversal and execution.

There is more to explore, including additional derivative rules, more complex graph structures, and the integration of gradients with the training and optimization pipeline. For now, the key milestone is having a mechanism that connects recorded computations to their corresponding gradients.

To be continued...

Chapter 6

Neural Network Components: Layers



6.1 Layer

A layer is a high-level abstraction that encapsulates a sequence of mathematical operations into a reusable component. It allows us to construct complex neural network architectures without explicitly writing every underlying operation or manually defining the backward graph for each layer.

This abstraction separates model architecture from the mathematical details of individual operations. As a result, we can modify the architecture and its hyperparameters while relying on the framework to handle the underlying computations and automatic differentiation.

Let's now examine how layers are implemented in this framework.

The Layer class is an abstract base class that defines the common interface for derived layers. Each concrete layer implements the required functions:

the layer class is a abstruct class i.e can't be instantiated then later layers are inheriting and overriding the functions like

- calculate: Calculates the forward pass only in eager mode
- forward: Constructs the layer's computation in the current graph context for graph-mode execution.
- backward: Retrieves the gradients required by the layer's parameters and registers the corresponding optimizer update operations. It does not explicitly construct the backward graph for the layer's mathematical operations; those gradients are handled by the autograd system.
- basic constructor and destructors and some axulary functions to intercept the parameters or hyperparameters.

Let's see it in practice

```
class Layer {
public:
    virtual ~Layer() = default;

    virtual std::vector<tf::tensor>
    calculate(std::vector<tf::tensor> inputs) = 0;
    virtual std::vector<tf::tensor*>
    forward(std::vector<tf::tensor> inputs) = 0;
    virtual void backward() = 0;
};
```

Listing 6.1: Creating layer class

Now lets take a look at each layer and how they're constructed

6.2 Layer: Dense

Mathematically Dense layer is very simple, it does matmul bias add, which is
$Y = X@W + b$

- X: input temsor of rank 2. axis-1 is feature axis-2 is batch or sample size.
- W: is weights tensor of rank 2 axis-1 is Dense unit/no or neuron axis-2 is no of incoming features.
- b: is a vector or matrix of rank 1. axis-1 is no of dense unit/ no of neuron.
- Y: output tensor of rank 2. axis-1 is no of feature and axis-2 is batch or sample size.

The idea behind Dense layer is do a linear transformation on input with weights tensor and add bias to it.

If we wanted to implemente a dense layer without any layer abstruction it would look something like

```
#include <tensor.h>

tf::tensor X, W, B, Y;

X.tf_create(18, 512); // no_of_input_feature(18), and sample_size(512)
W.tf_create(16, 18);  // dense_unit(16), and no_of_incoming_feature(18)
B.tf_create(18); // dense_unit(16);

Y = X.matmul(W).add(B);
```

Listing 6.2: Dense Function

First of all we can we that but if we think from bigger perspective then it would be very clear that. This approch is not that easy to scale. If we get to the training phase then we need to mannually handle gradients and update them. So that is the reason behind abstruction.

No Lets take a look at how to properly make the abstruction

6.2.1 Dense: Constructor

```
// file dense.hpp
#include <layer.hpp>

Class Dense : protected Layer{
    Dense(unsigned unit): dense_unit(unit){}

    std::vector<tf::tensor *>& operator(std::vector<tf::tensor>
   input_tensors);
    std::vector<tf::tensor> calculate(std::vactor<tf::tensor> inputs)
   override;
    std::vector<tf::tensor *>& forward(std::vector<tf::tensor> inputs,
   unsigned const batch_size) override;
    std::vector<tf::tensor *>& backward(Optimizer *const optimizer)
   override;
    ...
    // few other internal methods/functions
    ...
}
```

Listing 6.3: Dense constuctor and basic structure

simple easy peasy no rocket science. constructor is expecting the no of neuron for this layer. and assigning the vaue to its internal variable and few other internal flags, weights, bias initialization methods are set. Which is not shown here.

6.2.2 Dense: Overloaded () operator

```

// file dense.cpp
#include "dense.hpp"

const std::vector<tf::tensor *> &
Dense::operator()(std::vector<const tf::tensor> input_tensors){
    if(input_tensor.size() == 1){
        weights.tf_create();
        bias.tf_create();
        layer_outputs.push_back(tf::tensor(tf_float64));
    } else {
        // throw error, Dense layer expects only one input tensor
    }

}
```

Listing 6.4: Dense () overloaad

The overloaded operator() provides a convenient interface for connecting layers. During this stage, the layer prepares its parameters and output handles as required by the framework.

The initialization is lazy because the layer may not have enough information to initialize all its internal state when the constructor is called. The first build or forward operation provides the necessary context, such as the input shape and batch size.

The output handle is particularly important: it allows subsequent layers to reference the output of this layer without manually connecting every underlying mathematical operation.

Why use vectors of tensors instead of a single tensor? Although a Dense layer accepts one input tensor and produces one output tensor, the base interface supports layers that may consume multiple inputs or produce multiple outputs. This keeps the layer interface general without forcing every layer to use the same input/output cardinality.

6.2.3 Dense: forward() backward() calculate()

```
1
2  // file dense.cpp
3  #include "dense.hpp"
4
5  const std::vector<tf::tensor *> &
6  Dense::forward(std:;vector<const tf::tensor *> &inpyt, unsigned
       batch_size){
7
8        this->weight.reshape(arr);
9        this->training_weight.tf_create(arr, tf_float64);
10       this->grad_weight.tf_create(arr, tf_float64);
11
12       ...
13       // initialize bias
14       // initialize outputs
15       ...
16
17       training_output.push_back(training_input.matmul(training_weights)
       .add(training_bias));
18
19       return training_ouput;
20
21 }
22
23
24
25 const std::vector<tf::tensor *> &
26 Dense::calculate(std:;vector<const tf::tensor *> &inpyt, unsigned
       batch_size){
27 initializeParameters();
28 std::vector<tf::tensor> output;
29
30 output.emplace_back(inputs[0].matmul(weights).add(bias));
31
32 return output;
33 }
34
35 Dense::backward(Optimizer *optimizer){
36
37   if (!this->isGradRecorded) {
38     Graph *g = GraphManager::instance().getCurrentGraph();
39     graph_weight = g->getGradTensor(training_weight);
40     grad_bias =  g->getGradTensor(training_bias);
41     grad_input = g->getGradTensor(training_inputs[0]);
42     grad_output = g->getGradTensor(training_output[0]);
43
44     optimizer->createParameterUpdateGraph(training_weight,
       updated_weight, grad_weight);
```

```
45      optimizer->createParameterUpdateGraph(training_bias, updated_bias,
        grad_bias);
46
47      this->isGradRecorded = true;
48    }
49  }
```

Listing 6.5: Dense calculate

Lets discuss one at a time:

forward()

This function implements the graph-mode forward path and is primarily invoked during model training through fit().

At this stage, the framework has access to the execution context and the relevant input shapes. The layer can initialize its parameters, prepare the required tensors, and construct the computation using the framework's primitive operations.

The graph context is managed by the model's training procedure, so the layer does not need to create a separate graph context for itself.

The important distinction is that forward() constructs the computation graph; it does not necessarily evaluate the tensor operations immediately.

calculate()

This function implements the eager-mode forward path and is primarily used by predict() when immediate inference is required.

Unlike graph construction, eager execution evaluates the underlying operations as they are invoked and returns the resulting output tensors.

backward()

It retrieves the gradients associated with the weights, biases, inputs, and outputs from the current graph context, as required by the implementation. It then registers the parameter update operations with the optimizer.

The gradients of the layer's mathematical operations are computed by the autograd system. The layer's backward() method does not manually derive those gradients; instead, it exposes the relevant gradient tensors and prepares the optimizer update path.

The isGradRecorded flag prevents these registrations from being repeated unnecessarily within the intended graph lifecycle.

6.2.4 Dense in practice

```
1  // file dense_test_01.cpp
2  #include <tensor.h>
3
4  void main(){
5      tf::tensor inputs, W, B, ouputs;
6
7      inputs.tf_create(tf_float64, 19, 1000);
8      W.tf_create(tf_float64, 16, 19); // tensor to seed weight
9      B.tf_create(tf_tf_float64, 16);  // tensor to seed bias
10
11     input.tensor_of(-2.0, 3.0);
12     W.tensor_of(-1.0, 1.0);
```

```
    B.tensor_of(0.0, 0.0);

    auto dense = tf::layer::dense(16); // 16 is dense unit a.k.a no of
    neurons
    dense.set_weight(W);
    dense.set_bias(B);

    output = dense.calculate({inputs})[0]; // output holds the output
    from dense layer

}
```

Listing 6.6: Dense in practice

Dense Training

backward(), is already covered so, we know how it is done in code but let's touch upon mathematical construct of Dense backward

Weight updation

$$\bar{W} \leftarrow W - \nu * \delta W$$

$$\delta W = ZX^T$$

Bias updation

$$\bar{B} \leftarrow B - \nu * \delta B$$

$$\delta W = X$$

Gradient propagation

$$\delta X = W^T Z$$

The input gradient is propagated to the preceding layer; it is not updated using the learning rate. The optimizer updates trainable parameters, while autograd propagates gradients through the computational graph.

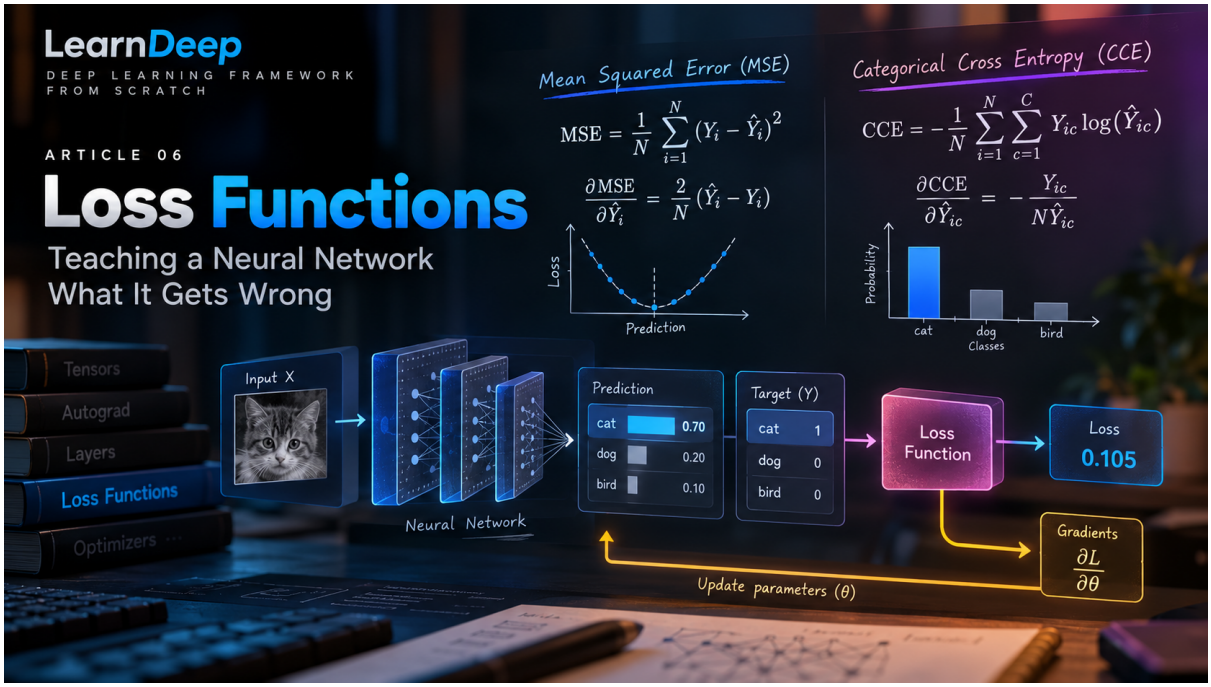
Where,

Z is incoming grad

X is input

Chapter 7

Neural Network Components: Losses



From the previous chapters, we've established the core components of our framework, including tensors, operations, autodiff, and neural network components such as layers.

So far, our framework is good at performing mathematical computations through forward and backward passes. But how does it know whether its predictions are correct? More importantly, how does it know whether training is making progress?

Our model's trainable parameters are initialized, and gradients can be calculated through the computational graph. But we still need a way to measure the difference between the model's predictions and the expected outputs.

That's where loss functions come in.

In supervised learning, a loss function measures the discrepancy between predicted outputs and their corresponding targets. During training, we typically aim to minimize this loss. For each batch, the model produces predictions during the forward pass, and the loss function evaluates how far those predictions are from the actual targets.

Let's see how this works in practice.

7.1 Loss Functions: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2$$

where N is the number of elements over which the squared errors are averaged, Y_i is the target, and $\hat{Y}_i$ is the predicted output.

For each element, we subtract the prediction from the target and square the resulting difference. We then calculate the mean of these squared errors over the selected elements.

7.1.1 MSE: Differentiation

Although our autodiff engine calculates the gradients automatically, let's look at the derivative to understand what happens during backpropagation.

Differentiating MSE with respect to a prediction gives:

$$\frac{\partial \text{MSE}}{\partial \hat{Y}_i} = \frac{2}{N}(\hat{Y}_i - Y_i)$$

This gradient measures how the loss changes with respect to each predicted value. Our autodiff engine propagates it through the computational graph to calculate gradients for the model's trainable parameters.

7.1.2 MSE: Putting things together

Okey, now as we've sorted out the math part, we can now focus on the implementation. If we think deeply losses share common traits from their bigger sibling layers. They have few differences

- 1. Don't have trainable parameter
- 2. Outputs a scaler value rather than tensor.

So, we can follow same structure like Layers

```
// file loss.hpp

class Loss {
public:
    virtual ~Loss() = default;

    virtual float calculate(std::vector<tf::tensor> inputs) = 0;
    virtual void forward(std::vector<tf::tensor> inputs) = 0;
    virtual void backward() = 0;
};
```

Listing 7.1: Creating loss class

Although the loss function does not need to implement differentiation manually, its backward interface can provide access to the gradients recorded by the computational graph. This is useful for debugging and inspecting the intermediate gradients.

Let's see how MeanSquaredError(MSE) is layed out.

```
// file mse.cpp
#include <mse.hpp>

    void MSE::forward(std::vector<tf::tensor> inputs) {
      if (inputs.size() == 1) {
```

```
        training_inputs.push_back(inputs[0]);
        ...
        // initialize the intermediate tensors
        ...

        differences = target_ouput.sub(training_inputs[0]);
        loss_tensor = differences.pow(2).scale(1.0/batch_size);
      }
    };
    void MSE::backward(){
        if (!this->isGradRecorded) {
            this->isGradRecorded = true;
            Graph *g = GraphManager::instance().getCurrentGraph();
            grad_training_inputs = g,getGradTensor(trining_inputs[0]);
        }
    };
```

Listing 7.2: Creating MSE class

but sadly I've don't have any standalone configuration to showcase how a user can use losses. Most of its usage are inside model. As of now simple eager execution is not ready yet.

7.2 Loss Function: Categorical Cross Entropy (CCE)

While MSE is commonly used for regression tasks, Categorical Cross Entropy (CCE) is commonly used for classification tasks. It measures how well the predicted class probabilities match the target distribution.

For one-hot encoded targets, CCE is defined as:

$$\text{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} Y_{ic}\log(\hat{Y}_{ic})$$

where N is the batch size and C is the number of classes.

For each sample, we multiply the target value for each class by the logarithm of its predicted probability, sum over the classes, and average over the batch. The negative sign ensures that minimizing the loss encourages the model to assign higher probabilities to the correct classes.

7.2.1 CCE: Differentiation

$$\frac{\partial CCE_{\text{batch}}}{\partial(\hat{Y})} = -\frac{1}{n}\frac{Y}{\hat{Y}}$$

7.2.2 CCE: Putting things together

```
// file cce.cpp
#include <cce.hpp>

    void CCE::forward(std::vector<tf::tensor> inputs) {
      if (inputs.size() == 1) {
        training_inputs.push_back(inputs[0]);
        ...
        // initialize the intermediate tensors
        ...

        log_differences = target_inputs[0].mul(training_inputs[0].log()
   );
```

```
12          loss_tensor = log_differences.scale(1.0/batch_size);
13        }
14    };
15    void CCE::backward(){
16        if (!this->isGradRecorded) {
17            this->isGradRecorded = true;
18            Graph *g = GraphManager::instance().getCurrentGraph();
19            grad_training_inputs = g,getGradTensor(trining_inputs[0]);
20        }
21    };
```

Listing 7.3: Creating CCE class

7.2.3 CCE: In practice

Like MSE I've don't have eager mode calculate() is still not ready. So I don't have much ex. to showcase.

Chapter 8

Neural Network Components: Optimizers



We've already established the basic workings of an end-to-end neural network. We've seen how tensors, operations, graphs, layers and losses come together to perform a sequence of mathematical operations that produce the intended output.

However, measuring how far the prediction is from the desired result is only part of the process. As discussed in Chapter 7, the loss function quantifies the prediction error. We still need a way to minimize this error by updating the model parameters. This is where optimizers come into play.

During initialization, the model parameters are typically assigned values using an initialization strategy. These initial values generally do not produce satisfactory predictions. During training, the optimizer uses the gradients calculated during backpropagation to update the model parameters, helping the network minimize its loss.

We've already seen that the model parameters are initialized randomly. So at first it is going to give very incorrect output. Now it is optimizer which makes changes to the model parame-

ters based on the gradients calculated during backward pass that pushes model parameters to calculate output close to desired target. Lets see how it works.

8.1 Gradient Descent: GD

Gradient descent is one of the fundamental optimization algorithms used to train neural networks. It updates model parameters in the direction opposite to the gradient of the loss function.

Compared with second-order methods such as Newton-Raphson, standard gradient descent is generally simpler to implement and requires less memory because it does not need to compute and store the full Hessian matrix. This makes it a practical choice for large neural networks.

$$W_{\mathrm{t}+1} = W_{\mathrm{t}} - \eta * \partial W_t$$

from the above example we can easily tell what a optimizer suppose to do. For a model parameter W we're updating it's value by subtracting partial difference times η with original value. η is learning rate.

Learning Rate The learning rate, denoted by η, controls the step size of each parameter update.

If the learning rate is too large, the optimizer may overshoot the minimum, oscillate around it, or even diverge. If it is too small, convergence may become unnecessarily slow.

Choosing an appropriate learning rate is therefore critical to effective training. In practice, the learning rate may also be adjusted during training using a learning-rate schedule.

We're not going to work on a single sample. So working with a large sample size. Based on how we can tackle a large sample size and update model parameter values there are three distinct GD stratigies in place.

- Batch Gradient Descent(BGD)
- Mini Batch Gradient Descent(MGD)
- Stocastic Gradicnt Descent(SGD)

Now, Lets take a look at each of them.

8.1.1 Batch Gradient Descent: BGD

The gradient is calculated using the entire training dataset, and the parameters are updated once after the full dataset has been evaluated.

Pros: The gradient represents the entire dataset, giving a consistent update direction for the current parameter values.

Cons: Each update can be computationally expensive for large datasets.

8.1.2 Mini Batch Gradient Descent: MGD

The training dataset is divided into smaller batches. The model processes each batch and updates its parameters after each batch.

Pros: Balances computational efficiency and gradient stability. It also works well with vectorized CPU and GPU operations.

Cons: The gradient is an estimate of the full-dataset gradient, so updates can fluctuate between batches.

8.1.3 Stocastic Gradient Descent: SGD

In its strict definition, the gradient is calculated using one randomly selected training sample per update.

Pros: Updates are inexpensive and frequent. The noise in the gradient can sometimes help the optimizer move away from saddle regions or shallow local minima.

Cons: Updates can fluctuate considerably, making convergence less smooth.

8.2 Stocastic Gradient Descent

I've briefly touched SGD in the previous section but lets dig deeper into this algorithm. So I've already established how SGD works Mathematically. Let's see how it is integrated with the framework.

```
// file optimizer.hpp
class Optimizer{
    Graph *optimizer_graph;

  virtual void createParameterUpdateGraph(tf::tensor param,
                                          tf::tensor &update_param,
                                          tf::tensor gradient_param) =
   0;

  void executeOptimizer();
}
```

Listing 8.1: Creating Optimizer class

```
// file optimizer.cpp
#include "optimizers.hpp"
#include <core/graph/graph_framework.hpp>

Optimizer::Optimizer() { this->optimizer_graph = new Graph(); }
void Optimizer::executeOptimizer() { this->optimizer_graph->compute();
   }
```

Listing 8.2: Defining Optimizer class

So, Optimizer is a abstruct class which works as base class for other optimizers.

The createParameterUpdateGraph() subrountine is something new. Mostly for industry standard frameworks optimizers works in eager mode, in my case it is operating in graph mode. Why not to attempt something differet.

```
// file sgd.cpp
#include "optimizers.hpp"
#include <core/graph/graph_manager.hpp>

SGD::SGD() : learning_rate(0.01) {}

void SGD::createParameterUpdateGraph(tf::tensor param, tf::tensor &
   update_param,
                                     tf::tensor gradient_param) {
```

```
9
10    GraphManager::instance().pushGraph(this->optimizer_graph);
11    scaler_param.push_back(tf::tensor());
12
13    std::vector<unsigned> dims;
14    for (unsigned i = 0; i < param.getNoOfDimensions(); i++)
15      dims.push_back(param.getDimensions()[i]);
16    scaler_param.back().tf_create(dims, param.dt_type);
17
18    scaler_param.back() = gradient_param.scale(learning_rate);
19    update_param = param.sub(scaler_param.back());
20
21    GraphManager::instance().popGraph();
22  }
23
24  void SGD::updateLearningRate(std::float64_t learning_rate) {
25    this->learning_rate = learning_rate;
26  }
```

Listing 8.3: Defining SGD class

Here we can see GraphManager in action. By pushing the optimizer graph onto the graph manager, the operations used to calculate parameter updates are recorded in the appropriate graph.

This graph is constructed during the setup of the training process for each trainable parameter. Once a training batch has completed its forward and backward passes, the training loop executes the optimizer graph to calculate the parameter updates.

This separates the construction of the update operations from their execution during training.

Although vanilla SGD is simple and computationally efficient, it has some limitations. Its updates can fluctuate significantly, and it may converge slowly in regions where the loss landscape has poorly conditioned curvature.

Several optimizer variants address these issues by incorporating additional information into the parameter updates. One of the simplest and most important extensions is SGD with momentum, which we will explore next.

8.3 SGD with momentum

Chapter 9

Neural Network Model Training and Inference



So far, we have developed the fundamental components required to construct and train a neural network, including tensors, layers, loss functions, optimizers, and automatic differentiation.

The next step is to bring these components together into a complete training and inference workflow.

In this chapter, we will explore the public model API exposed by LearnDeep and understand how it orchestrates the underlying components. We will examine model construction, compilation, training, and inference, and discuss how the implementation can be verified against independently generated reference results using NumPy.

The objective is not to revisit the implementation details of each component, but to understand how they interact to form a functional neural network framework.

9.1 The Model Class

The Model class provides the high-level interface through which users define and execute a neural network. It abstracts the underlying tensor operations, computational graph, automatic differentiation, loss calculation, and parameter optimization behind a small set of public API functions.

The following listing presents the principal interface of the Model class.

```
class Model {
  void compile(const OptimizerType optimizer_type,
               const LossType loss_type);

  tf::history_container fit(
      const std::vector<tf::tensor>& inputs,
      const std::vector<tf::tensor>& outputs,
      const std::vector<tf::tensor>& validation_data = {},
      unsigned epochs = 10,
      unsigned batch_size = 1,
      std::vector<std::shared_ptr<Callback>> callback = {},
      unsigned verbose = 0,
      bool training = true);

  std::vector<tf::tensor> predict(
      std::vector<tf::tensor> inputs);

  void shuffle(bool shuffle);

  tf::loss getModelLoss(
      Tensor<std::float64_t>* output_tensor);
}
```

Listing 9.1: Public interface of the Model class

The API separates the definition of a model from the operations performed on it. Model construction establishes the relationship between input and output tensors, while compilation configures the loss function and optimizer. The fit() method coordinates training, and predict() provides an interface for evaluating the model on new inputs.

The remaining methods support training configuration and loss-related operations.

Let us examine how these responsibilities are implemented.

9.1.1 Model Construction and Compilation

Many neural network libraries provide a sequential interface in which layers are arranged in a predefined order. Each layer receives the output of the preceding layer and passes its own output to the next layer.

LearnDeep instead represents a model through its input and output tensors and the computational dependencies connecting them. This approach is not restricted to a sequential arrangement of layers.

For example, consider the following model construction:

```
tf::model mymodel({x}, dense_output);

mymodel.shuffle(false);

mymodel.compile(OptimizerType::SGD,
                LossType::squared_error);
```

Listing 9.2: Constructing and compiling a model

Here, x represents the model input, while dense_output represents the output tensor produced by the network. The model is constructed by associating these tensors, rather than by explicitly passing an ordered collection of layers to the Model constructor.

During model construction, LearnDeep traverses the dependencies of the declared output tensors to determine whether they can be traced back to the declared inputs. If the required dependencies cannot be resolved, the model construction is rejected.

This process establishes the computational path required to evaluate the model. It also provides the foundation for executing the forward computation and, during training, propagating gradients through the relevant operations.

The distinction between a sequential arrangement and a computational graph becomes particularly important when a model contains multiple inputs, branching paths, or operations that combine outputs from different parts of the network.

The supported graph structures ultimately depend on the operations and execution strategies implemented by LearnDeep.

Compilation Once the model has been constructed, compile() configures the components required for training.

The optimizer type determines the parameter update strategy, while the loss type determines how the model's predictions are evaluated against the target outputs.

For example, the configuration shown above selects stochastic gradient descent (SGD) as the optimizer and squared error as the loss function.

Based on these selections, LearnDeep creates the corresponding optimizer and loss instances. This separates the choice of training configuration from the implementation of the training loop itself.

The optimizer and loss function serve different purposes. The loss function quantifies the prediction error, whereas the optimizer uses the resulting gradients to update the trainable parameters.

Training Data Shuffling The shuffle() method controls whether the training samples are shuffled before being divided into mini-batches.

When shuffling is disabled, the samples retain their original order. When enabled, their order is randomized according to the implementation.

In the example above, shuffle(false) preserves the original sample order.

Shuffling is useful during training because it reduces dependence on the original ordering of the training samples. However, it does not change the underlying model architecture or the mathematical definition of its operations.

9.1.2 Model Training

Training is the process through which a neural network adjusts its trainable parameters to reduce the discrepancy between its predictions and the target outputs.

The fit() method provides the interface for executing this process.

```
unsigned epochs = 10;
unsigned batch_size = 128;

auto history = mymodel.fit(
    {input},
    {target_output},
    {},
    epochs,
    batch_size,
    callback);
```

Listing 9.3: Training a model

The training inputs and corresponding target outputs are provided as collections of tensors. The epochs argument specifies how many complete passes are performed over the training dataset, while batch_size determines the number of samples processed in an individual mini-batch.

During training, the framework coordinates the following operations:

1. **Mini-batch selection:** The training samples are divided into batches according to the configured batch size and shuffling policy.
2. **Forward propagation:** The input tensors are passed through the computational graph to calculate the model predictions.
3. **Loss calculation:** The predictions are compared with the corresponding target outputs using the configured loss function.
4. **Backward propagation:** Automatic differentiation computes the gradients of the loss with respect to the trainable parameters.
5. **Parameter optimization:** The optimizer uses the calculated gradients to update the model parameters.
6. **Iteration:** The process continues for the remaining mini-batches and repeats until the requested number of epochs has been completed.

The process connects the components developed in the previous chapters. Tensor operations perform the forward computation, the loss function evaluates the predictions, automatic differentiation calculates gradients, and the optimizer updates the parameters.

The fit() method therefore acts as the coordinator of the training workflow rather than implementing each of these operations independently.

Callbacks and Training History Callbacks provide a mechanism for observing or responding to events during training. Depending on the callback implementation, they can be used to record metrics, monitor training progress, or control the training process.

The fit() method returns a history container that preserves the training information recorded during execution. This allows the user to inspect the training results after the fit() call has completed.

The exact contents of the history depend on the metrics and callback behavior supported by LearnDeep.

The validation data argument can be used to provide additional data for evaluating the model during training, provided that validation evaluation is implemented by the training workflow. Validation data is distinct from the training data because it is used to evaluate model behavior rather than to perform gradient-based parameter updates.

9.1.3 Model Inference

After training, the model can be used to generate predictions for a given set of input tensors.

Inference evaluates the model using its current parameter values without performing optimizer-driven parameter updates.

The predict() method provides this interface.

```
auto predicted_output = mymodel.predict({input});
```

Listing 9.4: Generating predictions using a trained model

The input tensors are evaluated through the model's computational graph, and the resulting output tensors are returned to the caller.

Unlike training, inference does not require target outputs or a loss calculation for the purpose of updating model parameters. Its primary responsibility is to produce predictions from the supplied inputs.

When predict() is called on a model that has already been trained, it uses the model's current weights and biases. Consequently, the predictions reflect the parameter updates performed during training.

Separating training from inference provides a clear distinction between learning the model parameters and using those parameters to evaluate new data.

9.2 Numerical Verification

Implementing the model API is only one part of developing a neural network framework. Its numerical behavior must also be verified against a trusted reference.

LearnDeep uses NumPy to generate reference results for selected operations and model configurations. These results can then be compared against the outputs produced by the C++ implementation.

For a forward-pass test, the model is configured with known weights and biases, and its predictions are compared with the corresponding NumPy results. This helps verify that the model correctly combines its underlying tensor operations.

Training requires additional verification. The parameter updates and loss behavior should be examined independently to establish that gradients and optimization are working as intended.

Finally, inference can be tested by checking whether predict() returns the expected output tensors for a known model state and a given set of inputs.

These tests address different aspects of correctness: forward-pass tests verify numerical computation, training tests examine gradient-based optimization, and inference tests verify the prediction interface.

Together, they provide a systematic approach to validating the integration of the components developed throughout this framework.

$$\nabla_\theta L$$